

文件系统

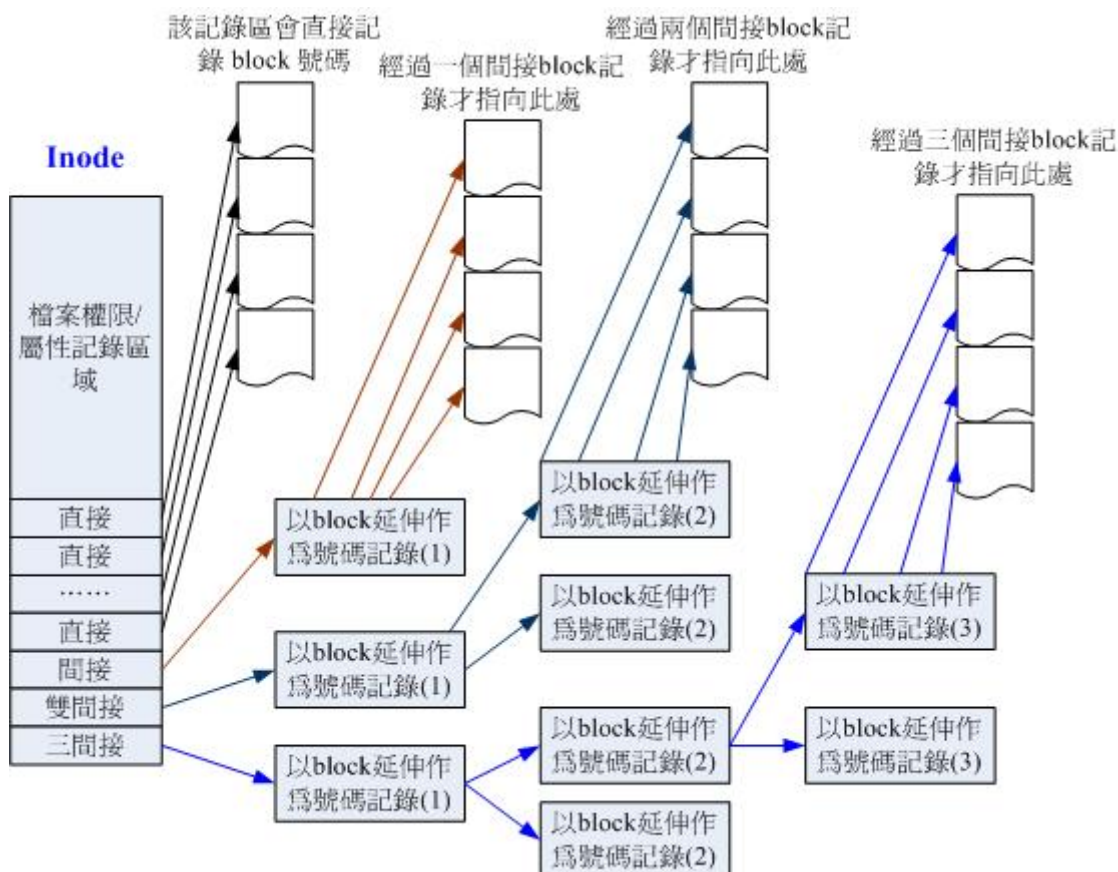
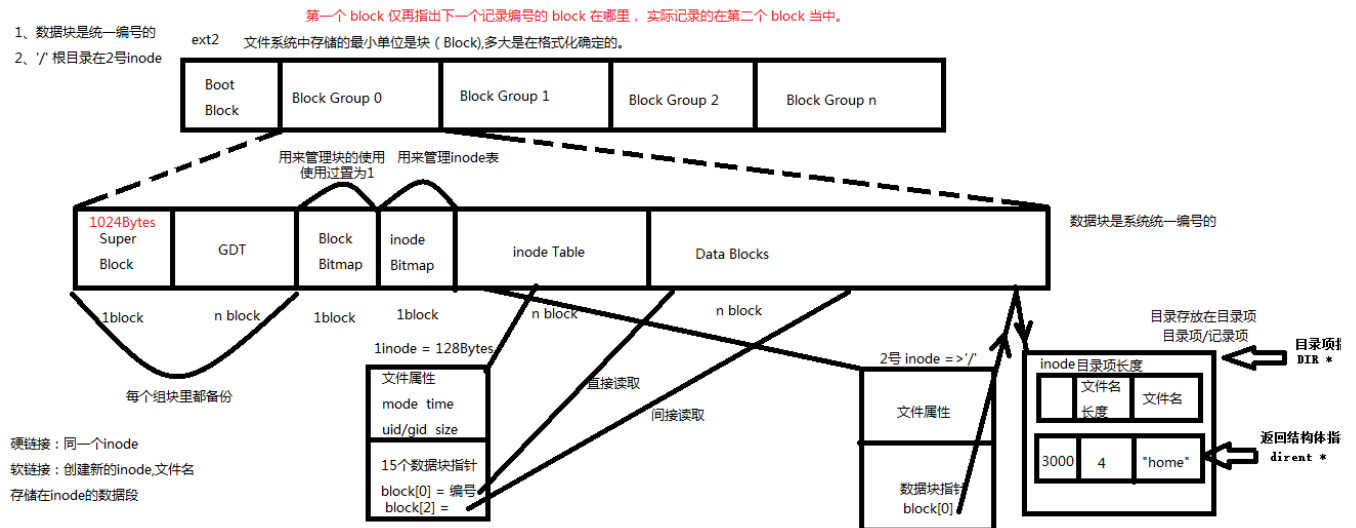
1、ext2文件系统

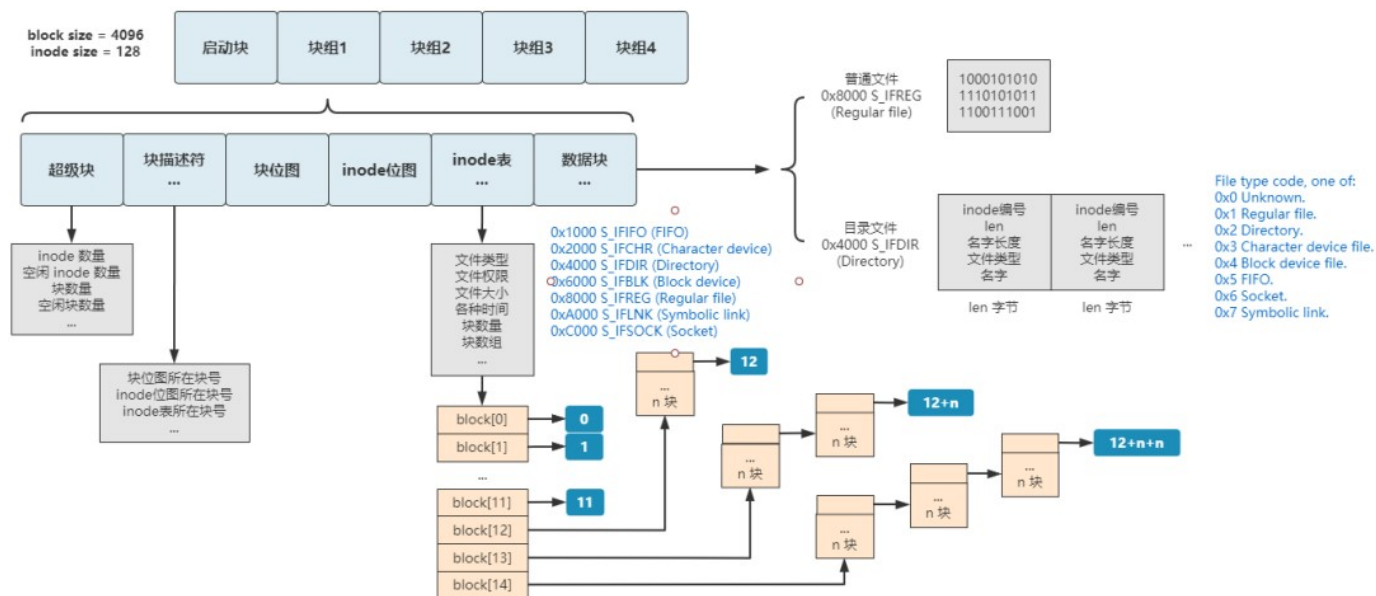
磁盘：512Bytes为一个扇区，2个扇区组成一个块（block）（设定）。1block = 1024Bytes.

- 1、**启动块（boot block）**用来存储磁盘分区信息和启动信息，大小为1KB.任何文件系统都不能使用启动块。
- 2、**超级块（Super Block）**描述整个分区的文件系统信息，如块的大小，文件系统版本号，上次mount的时间。超级块在每个块组的开头都有一份拷贝。每个块组都有一份（互为备份）。
- 3、**块组描述符表（GDT）**由多块构成描述符，整个分区分成多少个块组就对应有多少个块组描述符（互为备份）。每个块组描述符存储一个块组的描述信息，inode表从哪里开始，数据块从哪里开始，空闲的inode和数据块还有多少个等信息。（查看磁盘df，计算GDT）
- 4、**块位图（block bitmap）**用来描述整个块组中那个块被使用，每个bit代表块组的一个块。块位从超级块计算，使用置为1，未使用置为0。
- 5、**组块（group block）**有多少个块，取决于1block有多少位，就有多少个块，组块就有多大，组块从超级块计算，不包含启动块。 $1\text{ block} = 1024\text{Bytes} * 8 = 8196\text{bit}$
- 6、**Inode位图（inode Bitmap）**占用一个块的大小。用于标识被使用过的inode结构体。
- 7、**Inode 表（inode Table）**是由inode结构体，占用块而构成的一张表，存储着文件的属性。一部分存储文件属性，一部分存储数据块指针（15个共64字节），既是指向数据块的地址。一个inode结构体大小128Bytes.（删除一个文件，其实在inode位图置成零），**inode是操作系统统一编号**。
- 8、**数据块（data block）**存储数据的块，**数据块的编号是统一编号的**。
- 9、数据块寻址，有12指针用于直接寻址，有一级间接寻址，二级间接寻址，三级间接寻址，一个15个指针指向数据块。

10、文件类型：Unknown \Regular file \Directory \Character device \Block device \Name pipe \Socket \Symbolic link

11、如果是目录，那么对应的数据块存储的是目录项/记录项。目录项的结构体包含 inode号，文件名长度，文件名，目录项长度；（比如：home,文件名长度4，文件名“home”）。





1. 超级块前面是启动块，这个是 PC 联盟给硬盘规定的 1KB 专属空间，任何文件系统都不能用它。

2. ext2 文件系统首先将整个硬盘分为很多块组，但如果只有一个块组的话，和我们的文件系统整体结构就完全一样了，分别是超级块、块描述符、块位图、inode 位图、inode 表、数据块。

3. ext2 文件系统的 inode 表中用 15 个块来定位文件，其中第 13 个块为一级间接索引、14 个为二级间接索引、15 个为三级间接索引。

4. ext2 文件系统的文件类型分得更多，还有常见的如块设备文件、字符设备文件、管道文件、socket 文件等。

5. ext2 文件系统的超级块、块描述符、inode 表中记录的信息更多，但核心的和我们的文件系统一样，而且这些字段在后续的 ext3 和 ext4 中不断增加，保持向前兼容。

6. ext2 文件系统的 2 号 inode 为根目录，而我们的系统是 0 号 inode 为根目录，这个很随意，你设计一个文件系统定一个 187 号 inode 为根目录也没人拦着你。

如果你想了解 ext2 文件系统的全部细节，有三种方式。

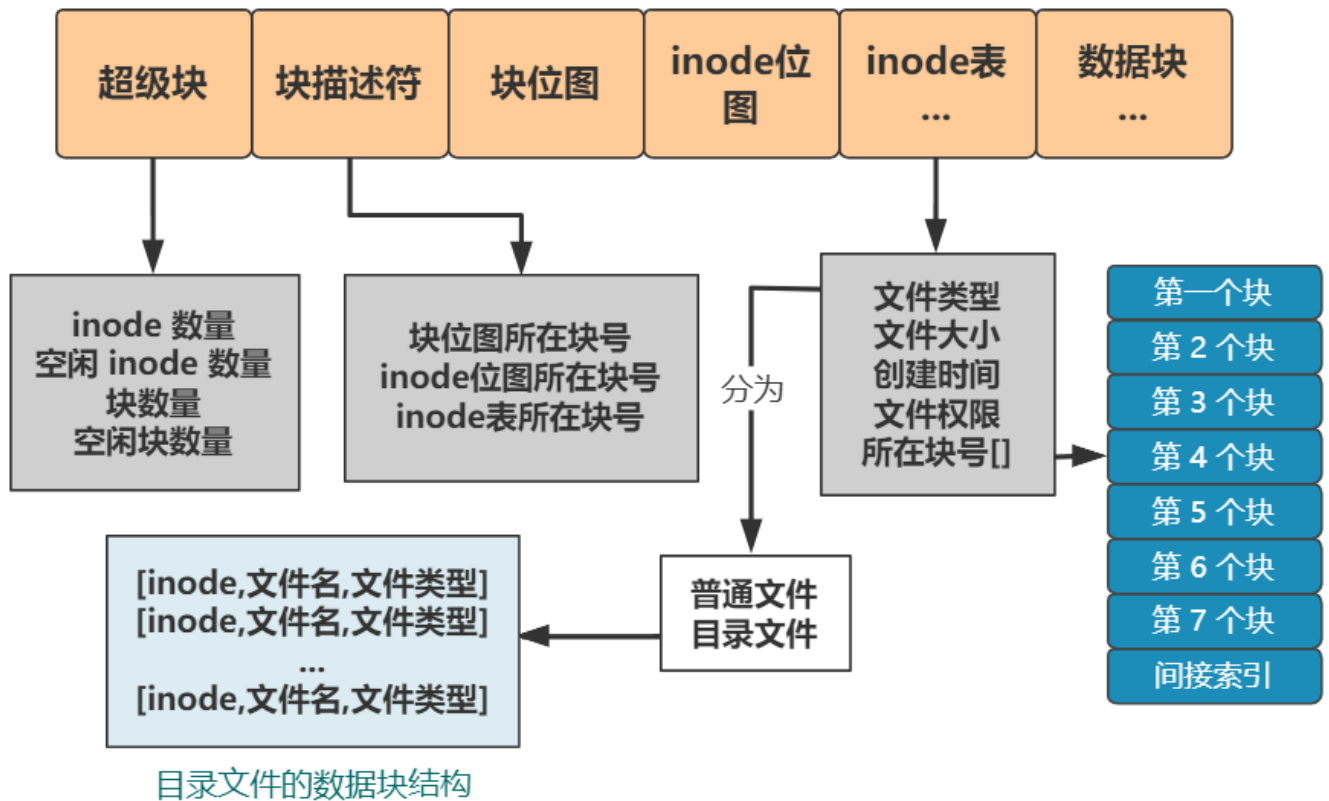
1. 看源码，linux1.0 后的源码都有 ext2 文件系统的实现，源码是最准确的。
2. 看官方文档，这里有个 pdf 连接。

<https://www.nongnu.org/ext2-doc/ext2.pdf>

3. 看优质博客，这里我推荐一个。

<http://docs.linuxtone.org/ebooks/C&CPP/c/ch29s02.html>

4. 用 linux 的 mke2fs 命令生成一个 ext2 文件系统的磁盘镜像，然后一个字节一个字节分析其格式，可以在公众号 低并发编程 回复 ext2 获得我的镜像分析文件。



2、stat函数（用来获取文件inode里的文件属性）

```
1 int stat (const char *path, struct stat *buf)
2
3 struct stat {
4     dev_t st_dev; /* ID of device containing file */
5     ino_t st_ino; /* inode number */
```

```
6 mode_t st_mode; /* protection */
7 nlink_t st_nlink; /* number of hard links */
8 uid_t st_uid; /* user ID of owner */
9 gid_t st_gid; /* group ID of owner */
10 dev_t st_rdev; /* device ID (if special file) */
11 off_t st_size; /* total size, in bytes */
12 blksize_t st_blksize; /* blocksize for file system I/O */
13 blkcnt_t st_blocks; /* number of 512B blocks allocated */
14 time_t st_atime; /* time of last access *////最后访问时间
15 time_t st_mtime; /* time of last modification *////修改文件内容的时间
16 time_t st_ctime; /* time of last status change *////文件属性改动时间
17 };
```

```
1 #include <sys/types.h>
2 #include <sys/stat.h>
3 #include <unistd.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6
7 void sys_err(char *str)
8 {
9     perror(str);
10    exit(1);
11 }
12
13 int main(int argc, char *argv[])
14 {
15     struct stat buf;
16     if(argc < 2)
17     {
18         printf("./a.out filename\n");
19         exit(1);
20     }
```

```

21
22     if(stat(argv[1],&buf) < 0)
23     {
24         sys_err("stat");
25     }
26
27     //判断文件类型
28     if(S_ISREG(buf.st_mode))
29     {
30         printf("- ");
31     }
32     else if(S_ISDIR(buf.st_mode))
33     {
34         printf("d ");
35     }
36
37     printf("%s : %d\n",argv[1],(int)buf.st_size);
38     return 0;
39 }

```

stat跟踪符号链接到原文件（原文件）。**Vim**打开文件有符号跟踪链接。**lstat**函数时不跟踪符号链接到原文件（也就是符号链接文件）。

符号链接的文件名存储在**inode**里。软链接不同一个**inode**，硬链接同一个**inode**。

每创建一个硬链接，**inode**的硬链接数增加1.在目录项里添加一条记录。

2、access函数访问文件是否有权限。

```

1 int access(const char *pathname,int mode)

```

按有效用户ID和有效组ID测试,跟踪符号链接

mode参数:

R_OK 是否有读权限

W_OK 是否有写权限

X_OK 是否有执行权限

F_OK 测试一个文件是否存在

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(int argc, char *argv[])
5 {
6     if(!access(argv[1], W_OK))
7     {
8         printf("%s W ok\n", argv[1]);
9     }
10    return 0;
11 }
```

```
1 #include <unistd.h>
2 #include <stdlib.h>
3 #include <stdio.h>
4
5 int main(int argc, char *argv[])
6 {
7     if(argc < 2)
8     {
9         printf("./a.out filename\n");
10        exit(-1);
11 }
```

```

11     }
12
13     if(access(argv[1],F_OK) != 0)
14     {
15         int ret = creat(argv[1],0664);
16         printf("argv[1] = %s ,%d F_OK\n",argv[1],ret);
17     }
18     return 0;
19 }

```

3、实际用户ID： 有效用户ID： sudo执行时，有效用户ID是root，实际用户ID是xingwenpeng

4、chmod函数（修改文件的权限位）mode_t 参数使用八进制数。前面应加零。0777。

```

xingwenpeng@ubuntu:~/LinuxCode/IOFile-02/kt$ chmod 06777 mytouch.sh
xingwenpeng@ubuntu:~/LinuxCode/IOFile-02/kt$ ls -l mytouch.sh
-rwsrwsrwx 1 xingwenpeng xingwenpeng 23  9月  4 09:37 mytouch.sh
xingwenpeng@ubuntu:~/LinuxCode/IOFile-02/kt$ ls
access.c  a.out  chmod.c  fstat.c  mls  mytouch.sh  stat.c

```

```

1 #include <sys/stat.h>
2 int chmod(const char *path,mode_t mode);
3 int fchmod(int fd,mode_t mode);

```

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 void sys_err(const char *str)

```



```

6 {
7     perror(str);
8     exit(-1);
9 }
10
11 int main(int argc, char *argv[])
12 {
13     if(argc < 2)
14     {
15         printf("./a.out filename\n");
16         exit(-1);
17     }
18
19     if(chmod(argv[1], 0777) != 0)
20     {
21         sys_err("chmod");
22     }
23
24     return 0;
25 }

```

5、chown函数（修改所有组）/etc/passwd（使用该函数必须是root权限）

```

1 注册名： 口令： 用户标识号： 组标识号： 用户名： 用户主目录： 命令解释程序
2  xingwenpeng:x:1000:1000:xingwenpeng:/home/xingwenpeng:/bin/bash

```

```

1 #include <unistd.h>
2
3 int chown(const char *path, uid_t owner, gid_t group);

```

```
4 int fchown(int fd, uid_t owner, gid_t group);
5 int lchown(const char *path, uid_t owner, gid_t group);
```

6、黏住位：在物理内存不够的情况下，加载多个进程，又想保留某个进程在物理内存中，而不让操作系统交换到虚拟内存，然而设置为黏住位，保证在物理内存中，而不被交换（希望）。

7、`/etc/shadow`（执行时，设置用户ID，来访问该文件）。`Sudo chmod 04777 filename`

8、`truncate`函数（拓展一个文件不需要跟一次写操作）（不会创建文件，需要文件存在）。

```
1 int truncate(const char *path, off_t length);
```

```
1 #include <unistd.h>
2 #include <sys/types.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 void sys_err(const char *str)
7 {
8     perror(str);
9     exit(-1);
10 }
11
```

```

12 int main(int argc, char *argv[])
13 {
14     if(argc < 3)
15     {
16         printf("./a.out filename size\n");
17         exit(-1);
18     }
19     int size = atoi(argv[2]);
20
21     if(access(argv[1],F_OK) != 0)
22         creat(argv[1],0664);
23
24     if(truncate(argv[1],size) < 0)
25         sys_err("truncate");
26
27     return 0;
28 }

```

9、link函数（创建一个硬链接）

```

1 #include <unistd.h>
2 int link(const char *oldpath, const char *newpath);

```

当rm删除文件时，只是删除了目录下的记录项和把inode硬链接计数减1，当硬链接计数减为0时，才会真正的删除文件。

硬链接通常要求位于同一文件系统中，符号链接没有文件系统限制。

通常不允许创建目录的硬链接，但某些unix系统可以在root用户创建目录硬链接。

创建目录项以及增加硬链接计数应当是一个原子操作。

目录一般是有两个硬链接数，第一个是目录名，第二个是“.”，多个层目录则有“..”。

”。

硬链接是存在同一个文件系统中，而软链接却可以跨越不同的文件系统。

硬链接：一个文件系统中有两个或更多个不同的文件名，都是对应同一个文件。

10、symlink函数（创建符号链接）

```
1 int symlink(const char *oldpath,const char *newpath);
```

11、readlink函数（读符号链接所指向的文件名字，不读文件内容）

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(int argc,char *argv[])
6 {
7     int len;
8     char buf[1024];
9
10    if(argc < 2)
11    {
12        printf("./a.out filename\n");
13        exit(1);
14    }
15
16    len = readlink(argv[1],buf,sizeof(buf));
17    buf[len] = '\0';
18    printf("size: %d\t%s ->%s\n",len,argv[1],buf);
19    return 0;
```

12、unlink函数

```
1 int unlink(const char *pathname)
```

- 1、如果是符号链接，删除符号链接；
- 2、如果是硬连接，硬链接数减1，当减为0时，释放数据块和inode；
- 3、如果文件硬链接数为0，但有进程已打开该文件，并持有文件描述符，则等该进程关闭该文件时，kernel才真正去删除该文件；
- 4、利用该特性创建临时文件，先open或creat创建一个文件，马上unlink该文件。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <fcntl.h>
6 #include <unistd.h>
7
8 int main(void)
9 {
10     char buf[1024];
11     int len;
12     int fd = open("tempfile", O_CREAT | O_RDWR, 0664);
13
14     unlink("tempfile"); //等到该进程结束，则删除该文件
15     write(fd, "helloworld", 10);
16     lseek(fd, 0, SEEK_SET);
17
```

```
18     len = read(fd,buf,sizeof(buf));
19     write(STDOUT_FILENO,buf,len);
20     close(fd);
21
22     return 0;
23 }
```

unlink也是shell的命令。

13、rename函数（文件重命名，修改的是目录项）

```
1 int rename(const char *oldpath,const char *newpath);
```

14、chdir函数（改变当前进程的工作目录）

```
1 int chdir(const char *path);
```

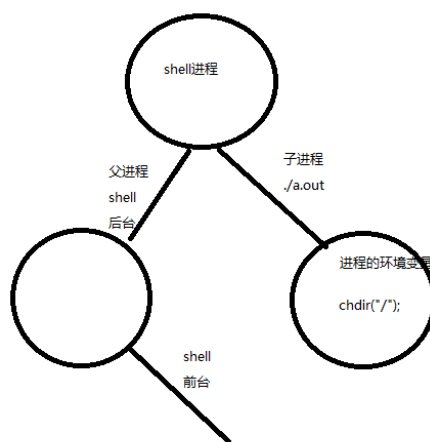
```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     char buf[1024];
7     getcwd(buf,sizeof(buf));//获取当前目录路径
8     printf("%s\n",buf);
9
10    chdir("/");
11 }
```

```

12     getcwd(buf, sizeof(buf));
13     printf("%s\n", buf);
14
15     return 0;
16 }

```

`chdir`属于环境变量，是在`shell`子进程中修改了路径，当子进程退出，又切换到父进程到前台，在父进程中的路径并未修改。 `pwd`



15、pathconf函数（测试文件名长度，系统中文件名长度）

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     printf("%ld\n", pathconf("abc", _PC_NAME_MAX));
7     return 0;
8 }

```

16、目录操作

1、mkdir函数

```
1 #include <sys/stat.h>
2 #include <sys/types.h>
3
4 int mkdir(const char *pathname, mode_t mode);
```

mode:权限位，创建目录需要执行权限

2、rmdir函数（删除空目录）

```
1 #include <unistd.h>
2
3 int rmdir(const char *pathname);
```

3、opendir/fdopendir函数（打开目录）

```
1 #include <sys/types.h>
2 #include <dirent.h>
3
4 DIR *opendir(const char *name);
5 DIR *fdopendir(int fd);
```

打开目录得到一个DIR指针，这个指针指向目录项

1、readdir函数（读取目录）


```

1 struct dirent *readdir(DIR *dirp); //返回一个目录项
2
3 struct dirent {
4
5     ino_t    d_ino; /* inode number */
6
7     off_t    d_off; /* offset to the next dirent */
8
9     unsigned short d_reclen; /* length of this record */
10
11     unsigned char  d_type; /* type of file; not supported by all file sy
12
13     char    d_name[256]; /* filename */
14 };

```

readdir每次返回一条记录项，**DIR***指针指向下一条记录项。

2、rewinddir函数（目录的读写指针，复位开始位置）

```

1 #include <sys/types.h>
2 #include <dirent.h>
3
4 void rewinddir(DIR *dirp);

```

把目录指针恢复到目录的起始位置。

3、telldir/seekdir函数

```

1 #include <dirent.h>
2
3 long telldir(DIR *dirp); //返回指针的位置
4

```

```
5 #include <dirent.h>
6
7 void seekdir(DIR *dirp, long offset); //设置目录指针的偏移量
```

4、closedir函数（关闭目录指针）

```
1 #include <sys/types.h>
2 #include <dirent.h>
3
4 int closedir(DIR *dirp);
```

17、readdir.c获取当前目录下的文件

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <dirent.h>
4 #include <stdlib.h>
5
6 void sys_err(char *str)
7 {
8     perror(str);
9     exit(1);
10 }
11
12 int main(void)
13 {
14     DIR *dp; //目录项指针
15     struct dirent *dirp; //每条目录项信息
16     dp = opendir("."); //打开当前目录
17     //if(dp == NULL)
```

```

18
19     if(!dp)
20         sys_err("opendir");
21
22     while((dirp = readdir(dp)) != NULL)//读当前每条目录项
23     {
24         printf("inode = %d\tname=%s\n",(int)dirp->d_ino,dirp->d_name);
25     }
26
27     closedir(dp);
28     return 0;
29 }

```

18、遍历目录下的文件

```

1 #include <sys/types.h>
2 #include <sys/stat.h>
3 #include <unistd.h>
4 #include <dirent.h>
5 #include <stdio.h>
6 #include <string.h>
7
8 #define MAX_PATH 1024
9
10 void dirwalk(char *dir,void (*fcn)(char *))
11 {
12     char name[MAX_PATH];
13     struct dirent *dp;
14     DIR *dfd;
15
16     if((dfd = opendir(dir)) == NULL)
17     {

```

```

18     fprintf(stderr, "dirwalk: can't open %s\n", dir);
19     return;
20 }
21
22 while((dp = readdir(dfd)) != NULL)
23 {
24     if(strcmp(dp->d_name, ".") == 0 || strcmp(dp->d_name, "..") == 0)
25         continue;
26     //home/yzmin / hello.c '\0'
27     //          1          2
28     if(strlen(dir)+strlen(dp->d_name)+2 > sizeof(name))
29         fprintf(stderr, "dirwalk: name %s %s too long\n", dir, dp->d_
30     else
31     {
32         sprintf(name, "%s/%s", dir, dp->d_name);
33         (*fcfn)(name); //fcfn(name)
34     }
35 }
36 closedir(dfd);
37 }
38
39 void fsize(char *name)
40 {
41     struct stat stbuf;
42     if(stat(name, &stbuf) == -1)
43     {
44         fprintf(stderr, "fsize: can't access %s\n", name);
45         return;
46     }
47
48     if((stbuf.st_mode & S_IFMT) == S_IFDIR)
49         dirwalk(name, fsize); //回调函数，fsize作为参数传递
50     printf("%8ld %s\n", stbuf.st_size, name);
51 }
52

```

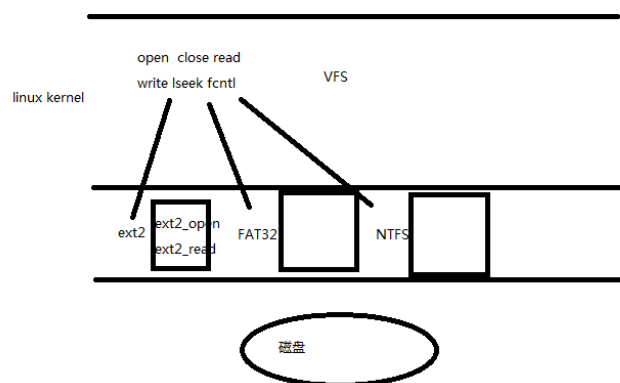
```

52
53
54
55 int main(int argc, char *argv[])
56 {
57     if(argc == 1) //当前目录
58         fsize(".");
59     else
60         while(--argc > 0) // ./a.out /home/yzmin
61             fsize(*++argv);
62
63     return 0;
64 }

```

19、VFS虚拟文件系统

VFS虚拟文件系统是为了支持多种文件格式，如：ext2,ext3,FAT,NTFS,iso9660等等，在linux内核中做了一个抽象层，使得文件，目录，读写访问等概念成为抽象层的概念。



20、dup/dup2函数

```

1 #include <unistd.h>

```

```

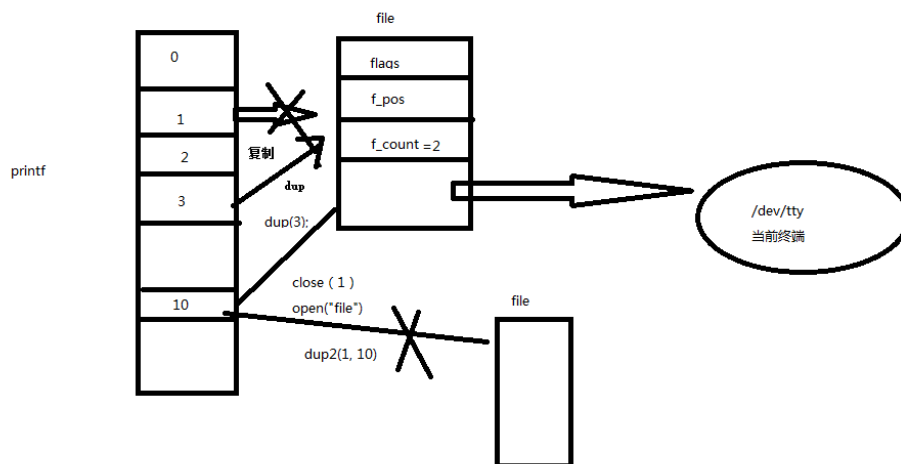
2
3 int dup(int oldfd); //复制一个文件描述符
4
5 int dup2(int oldfd, int newfd); //复制oldfd文件描述符，让newfd指向oldfd

```

对dup/dup2的应用:

dup和dup2都可用来复制一个现存的文件描述符，使两个文件描述符指向同一个file结构体。如果两个文件描述符指向同一个file结构体，File Status Flag和读写位置只保存一份在file结构体中，并且file结构体的引用计数是2。

如果两次open同一文件得到两个文件描述符，则每个描述符对应一个不同的file结构体，可以有不同的File Status Flag和读写位置。



1、dup.c

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4 #include <sys/stat.h>
5 #include <fcntl.h>
6
7 int main(void)

```

```

8 {
9     int fd,fp;
10    fd = dup(STDOUT_FILENO);//复制STDOUT_FILENO, 3
11    close(STDOUT_FILENO);//1
12
13    fp = open("abc",O_RDWR);//1
14    write(STDOUT_FILENO,"hello\n",6);//1
15    close(STDOUT_FILENO);//1
16
17    dup(fd);//1
18    write(STDOUT_FILENO,"world\n",6);
19
20    return 0;
21 }

```

2、dup2.c

```

1 #include <unistd.h>
2 #include <sys/stat.h>
3 #include <fcntl.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 int main(void)
9 {
10    int fd,save_fd;
11    char *msg = "This is a test\n";
12    fd = open("somefile",O_RDWR|O_CREAT,S_IRUSR|S_IWUSR);//3
13
14    if(fd < 0)
15    {
16
17    }

```

```

16     perror("open");
17     exit(1);
18 }
19
20 save_fd = dup(STDOUT_FILENO); //4
21 dup2(fd, STDOUT_FILENO); //1
22 close(fd); //3
23
24 write(STDOUT_FILENO, msg, strlen(msg));
25 dup2(save_fd, STDOUT_FILENO); //1
26 write(STDOUT_FILENO, msg, strlen(msg));
27 close(save_fd); //4
28
29 return 0;
30 }

```

